ENGINEERING REVIEW · ETA-TR-2026-001

Autonomous Exploratory Testing Agent System Capabilities & Code Review

What the system does, where every piece of it lives, the design decisions behind it, and a full-codebase audit of what is still wrong.

28 August 2026 Branch `raihan-working` 14,490 LOC Python · 74 endpoints · 24 graph node types

1 · CAPABILITIES

What the system does

> Six capabilities, each independently useful and each with a concrete artefact you can point at. The system reads a specification, builds its own map of a live application, writes tests traceable to requirements, runs them on a real device, learns from what happened, and reports with failures correctly attributed.

CAPABILITY	WHAT IT ACTUALLY DOES	EVIDENCE IT WORKS
Specification ingestion	Any text/PDF/DOCX spec → `Requirement`, `ValidationRule` and `Entity` nodes via multi-pass LLM extraction with per-section synthesis and self-critique. Format-agnostic: it knows nothing about numbering conventions.	99.9% text retention; 96 requirement ids from a 9k-char spec
Self-built UI map	Observes the running device and builds `UIState` nodes joined by `TRANSITIONS_TO`, deduped by a four-stage identity cascade. Needs no design file.	12 states / 44 transitions discovered autonomously
Requirement-traceable generation	A LangGraph loop retrieves from the graph over 2–3 rounds, then emits one test as JSON with explicit `COVERS` edges to the requirements it exercises.	25 tests per campaign; semantic dedup rejects rewordings
Real-device execution	Translates the test into a natural-language goal for a vision-language device agent driving a physical device or emulator, capturing a full step-by-step trajectory.	98 executions retained as on-disk trajectories
Experiential learning	Six layers fed by one executor write: execution logs, navigation memory, error patterns, strategy effectiveness, coverage heatmap, anomaly alerts.	123 nav nodes / 95 avoid; 3 mined error patterns; 1 unprompted anomaly alert
Attributed reporting	Every run classified into eight categories so application defects, agent limitations and environment problems are never summed. Exports CSV + review sheet.	100% autonomy measurable; 0 silent fallbacks

2 · CODE MAP

Where every piece lives

> Four processes. The planner and executor never talk to each other — everything they share goes through Neo4j, which is what makes the learning loop structural rather than incidental.

PATH	LOC	RESPONSIBILITY
KNOWLEDGE SERVICE — port 9010, sole owner of Neo4j		
rag_api/main.py	2,835	52 endpoints; ingestion, hybrid retrieval (vector + keyword + RRF), the state-identity cascade, app model, coverage
rag_api/learning.py	353	Error patterns, strategy memory, coverage heatmap, sessions, knowledge decay
rag_api/defects.py	381	Defect ingestion, density weighting, prone-area ranking
rag_api/anomalies.py	241	Failure-rate spikes, time regressions, new error types, path instability
rag_api/navtree.py	220	Shortest-path retention, avoid flags, cross-test reuse
rag_api/dimensions.py	179	Profile/platform/application partitioning and transfer
rag_api/risk.py · metrics.py · embeddings.py	~400	Regression risk scoring, effectiveness metrics, embedding backends
PLANNER — decides what to test		
planner/langgraph_agent.py	561	The retrieve → generate → dedup state graph
planner/prompts.py	494	Prompt assembly, session constraints, output contract
planner/pipeline.py	434	Ingestion orchestration, tiered-model selection
planner/context_builders.py	315	Learned-context and failure-context blocks
planner/budget.py	121	Priority-first token budget; P0 never dropped
planner/sources/	~300	Pluggable source registry — srs, live_ui, defects, navtree, figma
EXECUTOR — drives the device		
clients/executor_runner.py	1,344	Goal building, device agent, failure classification, livelock detection, observation recording, CSV export
SHARED		
ingestion/extractor.py	595	Multi-pass extraction, synthesis, self-critique, candidate selection
ingestion/app_state.py	372	Structural signature, Jaccard, containment, perceptual hash, labelling
settings.py	317	Single source of truth for every tunable
observability/	~350	Structured logging, tracing, metrics, degradation tracking
gateway/main.py	637	22 endpoints; thin router + dashboard data
dashboard-react/src/	811	Live app model graph, execution paths, planner trace, intelligence panels

3 · DATA MODEL

24 node types, three origins

ORIGIN	NODE TYPES
From the specification	`Requirement` · `ValidationRule` · `Entity` · `Chunk` · `SRS` · `SRSVersion` · `FeatureArea`
From the device	`UIState` · `UIElement` · `FigmaScreen`
From experience	`ExecutionLog` · `NavTreeNode` · `ErrorPattern` · `StrategyMemory` · `CoverageHeatmap` · `Session` · `AnomalyAlert` · `Defect` · `TestCase` · `TestRun`
Partitioning	`Project` · `Profile` · `Platform` · `Application`

Traceability runs on explicit `COVERS` edges from `TestCase` to `Requirement`. That single edge type is what lets requirement coverage be a first-class metric instead of a proxy such as activity or code coverage — the distinguishing property of this system relative to the GUI-testing literature.

4 · DECISIONS TAKEN

Design decisions, and what each one cost

> Every decision below bought something and gave something up. The tradeoff line matters more than the decision — it is where the next problem will come from.

1 · The graph is the only channel between planner and executor

Neither process calls the other. Every LLM call is stateless; the prompt is rebuilt from Neo4j from scratch each time. "Learning" means the graph grew, never that a model remembered.

`BOUGHT` One place where experience enters, so learning layers cannot drift apart. The device driver is replaceable without touching the planner.
`COST` Every signal must be modelled as a node before it can influence anything. Nothing can be passed informally.

2 · State identity is structural, not visual

A screen is reduced to package, activity, dialog flag and the set of controls keyed by `(resource_id, class, content-description, clickable)`. Text, list contents and scroll position are discarded.

`BOUGHT` Scrolling a list and switching to dark mode are both correctly the *same* state, for free. A vision model would call a dark screen a new screen.
`COST` Frameworks that expose a thin accessibility tree degrade the signature. On Flutter, `resource-id` appears on 1 node in 35 and `text` on none — identity rests almost entirely on `content-description`.

3 · A four-stage identity cascade, with containment as our own addition

Exact signature → Jaccard ≥ 0.90 → containment ≥ 0.95 → perceptual hash. Jaccard is $|A \cap B| / |A \cup B|$ and penalises a screen captured mid-render; containment is $|A \cap B| / \min(|A|, |B|)$ and scores that same subset 1.00.

`BOUGHT` Mapped states on an identical workload fell from 42 to 12 — the extra 30 were duplicates of screens already visited.
`COST` Containment can merge two genuinely different screens when one is a strict subset of the other. See finding A-5.

4 · Eight-way attribution before any measurement

Only `ASSERTION_FAILURE` and `CRASH` are evidence about the application. Agent failures lower autonomy; environment failures are excluded entirely.

`BOUGHT` The single thing that makes any number quotable. "The test failed" and "the agent got lost" stop being the same event.
`COST` A backend 500 currently has nowhere to go and lands in the application bucket. See finding A-6.

5 · One global prompt budget, filled by priority

A dozen independent caps were replaced by a single token ceiling filled highest-value-first. The bug oracle and on-screen controls are priority 0 and never dropped.

`BOUGHT` Nothing important is dropped while something unimportant is still present. Ceiling moved from ~8k to 50k tokens.
`COST` When the budget does bind, excess is discarded rather than summarised — a test can be regenerated once its title ages out of the dedup list.

6 · Tiered models: pay once for what everything depends on

Extraction runs on a large model; the per-test planner loop on one ~30× cheaper; the device agent on a mid-size vision model.

`BOUGHT` ~1¢ per test executed. Planning is under half a cent.
`COST` Extraction became the dominant line item at ~$1.80 per document on the flagship tier — 200 test executions to re-read one file. Now downgraded and capped to a few cents, with per-document caching still unimplemented.

7 · Degrade, but never quietly

One malformed screen must not end a two-hour run, so fallbacks stay — but each records what capability was lost, at what severity, surfaced on the dashboard and in the end-of-run report.

`BOUGHT` "Zero silent fallbacks" became a statement that means something.
`COST` Only the sites that were instrumented are covered. The audit found 78 exception handlers that still swallow silently, four of them consequential.

8 · No knowledge source is a hard dependency

A pluggable registry where each source implements the same three methods, and a bug-oracle cascade that bottoms out in generic heuristics.

`BOUGHT` The agent runs on an app with a spec, a UI guide, defect history, any subset, or none at all.
`COST` Correctness signal degrades quietly as sources are removed; there is no measure of how much oracle strength was lost.

9 · Credentials go to the executor, role goes to the planner

Recent addition for apps gated behind a login the agent cannot perform. The device agent receives the password because it types it; the planner receives only the role name.

`BOUGHT` A secret never enters the 50k-token planning prompt that is dumped, budgeted and traced everywhere.
`COST` The secret is still sent to a third-party model provider inside the device goal. Acceptable for a provisioned test account; not for a real one.

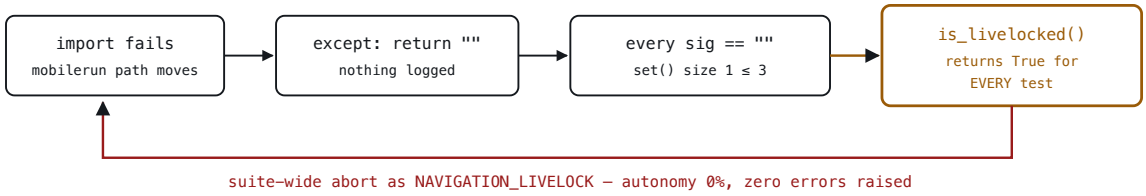
`5 · CODE AUDIT`

Full-codebase audit

> Every Python file parsed to an AST and checked for defect classes rather than read selectively — 14,490 lines across 56 modules. Findings are ordered by consequence, and the first one is reproducible on demand.

A-1 · Livelock detection inverts under an import failure `critical`

`clients/executor_runner.py:512, 539` — `_state_signature()` and `_content_fingerprint()` both import from `mobilerun` inside a `try` and **return an empty string on any exception**. If that import path ever changes, every observation yields `""`, and empty strings are all equal.



Verified by execution: with twelve empty signatures and twelve empty content fingerprints, `is_livelocked()` returns **True**. Thresholds are window 12, unique 3, content 2.

The failure presents as "*the agent got much worse*" rather than as a crash, which is the hardest possible shape to diagnose — it is the same signature as the reasoning-model incident that cost a full campaign. **Fix:** return a sentinel that cannot compare equal (a per-call UUID) rather than `""`, and record a degradation. An unusable signature must never look like a stable one.

A-2 · The Live App Model can switch itself off `high`

`clients/executor_runner.py:596` — `_record_observations()` begins with a `try: from mobilerun.macro.state import normalize_ui_state / except: return []`. That one handler disables the entire app-model build: no `UIState` nodes, no transitions, no navigation memory, and therefore no data for REQ-302 or REQ-303. The run still reports success. This is the identical shape to the `except: pass` that made requirement coverage read 0% for days.

A-3 · Learned navigation degrades invisibly `medium`

`clients/executor_runner.py:454` — a failed `/navtree/retrieve-path` call returns `""`, so the executor silently proceeds without the proven route. REQ-302's central benefit — fewer steps on repeat visits — disappears with no signal, and the only symptom is a step count that quietly stops improving.

A-4 · Observations dropped mid-loop `medium`

`clients/executor_runner.py:607, 614` — `continue` on a normalisation or base64 failure drops that observation from the app model. Partial, silent data loss: the resulting transition graph has holes that are indistinguishable from screens the agent never reached.

A-5 · Containment has no size-ratio guard `medium`

`rag_api/main.py:1899` — the guard requires only that the *smaller* control set has ≥ 8 members. An 8-control screen whose controls all also appear on a 42-control screen scores containment 1.00 and merges. Bucketing by package/activity/dialog limits the blast radius, but two genuinely different views within one activity can still collapse.

This is the inverse of the fragmentation bug we fixed, and the more dangerous direction: state fragmentation inflates a count and is obvious, whereas a false merge **hides a screen** — any defect reachable only there becomes structurally undiscoverable. **Fix:** require the larger set to be within a

bounded ratio of the smaller (a configurable `STATE_CONTAINMENT_MAX_RATIO`), and log every containment merge so the decision is auditable rather than silent.

A-6 · The taxonomy has no bucket for a backend failure `medium`

All eight categories describe the app, the agent or the device. A server-side 500 is none of those. Against Contacts this never arose — it is offline. Against a networked marketplace every backend hiccup will currently land in `ASSERTION_FAILURE` and inflate the candidate-defect count, which is precisely the measurement error the taxonomy exists to prevent.

A-7 · Structural `low`

Four functions exceed 220 lines — `build_testcase_prompt` (271), `execute_test_on_device` (262), `ingest_figma` (244), `_write_entity_graph` (228). None is defective, but each is long enough that the bugs we have already found in this project would hide comfortably inside them, and none is unit-testable in its current shape. The project has exactly one test file (`tests/test_app_state.py`, 11 checks) covering the one module that was easy to isolate.

Verified clean

CLASS	RESULT
Cypher injection	`clean` — the 9 dynamically-built queries interpolate only fixed dict lookups (`_DIM_LABEL`, `_DIM_REL`) and iterate a fixed `DIMENSIONS` tuple; every value is bound as `$param`
Bare `except:`	`clean` — zero occurrences
Mutable default arguments	`clean` — zero occurrences
Division by zero	`clean` — all four `/ len(...)` sites guarded
Unbounded HTTP calls	`clean` — every `requests` call carries a timeout
Shared mutable globals	`clean` — the 23 module-level mutables are constant lookup tables; the 7 `global` statements are lazy-init caches
Wasted judge call at N=1	`clean` — `_select_best` short-circuits on a single candidate

6 · STILL TO DECIDE

Decisions the system still needs

> Six open questions, ordered by how much they limit what we can claim. The first is not a bug but a policy, and it is the root cause of every serious defect found in this project.

> **1 · What is the standing policy on silent degradation?**
>
> Every major defect this project has produced — 86% of a spec discarded, coverage reading 0%, a device reset that reset nothing, and findings A-1 through A-4 above — is the same bug: an exception handler that made the system continue with less capability and said nothing. The degradation tracker fixed the sites we knew about; the audit shows 78 handlers still swallow silently.
>
> PROPOSAL Make silence the exception, not the default. Every `except` that changes behaviour records a degradation, and a run with a critical degradation refuses to report results as trustworthy. Enforce it in review rather than by remembering.

2 · How should an unusable signal be represented?

A-1 exists because "I could not compute this" and "I computed this and it was empty" share a representation. That pattern recurs: empty strings, empty lists and zeros all mean both *absent* and *genuinely nothing*.

PROPOSAL Unavailable is `None` or a non-comparable sentinel, never a falsy value of the normal type. Any comparison that could silently succeed on absent data becomes a type error instead.

3 · Should the taxonomy separate the system under test from its backend?

A-6. Adding `BACKEND_ERROR` is easy; deciding what it *means* is not. A 500 is a real defect in the product, just not one the UI caused — so it should probably be reported as a defect in its own class rather than excluded like an environment failure.

4 · How do we prove state identity is correct rather than merely stable?

We currently observe that state count fell from 42 to 12 and infer improvement. That is consistent with correct merging and equally consistent with over-merging (A-5). There is no ground truth for how many distinct screens the app actually has.

PROPOSAL Label the screens of one application by hand and measure the cascade against it. Fifty labelled observations would convert a plausible story into a number, and would price the containment threshold instead of guessing it.

5 · What is the testing strategy for the agent itself?

One test file, 11 checks, on 14,490 lines. We are building a testing tool that is largely untested, and the four longest functions are the ones carrying the most logic.

PROPOSAL Extract the pure decision functions — attribution classification, livelock detection, budget fitting, the identity cascade — and test those. They are where the damaging bugs have actually been, and all four are pure and cheap to test. A-1 would have been caught by a single test asserting that unusable signatures do not compare equal.

6 · When does requirement coverage become the headline metric?

Requirement traceability is the property that distinguishes this work from the GUI-testing literature, and it currently reads 2 of 33 — the mechanism works but few generated tests cite ids. Until that number rises, the differentiator is asserted rather than demonstrated.

PROPOSAL Make citing a requirement id mandatory in the output contract when the requirements block is non-empty, and treat an uncited test as a generation failure rather than accepting it.

Audit performed by AST traversal of all 56 Python modules (14,490 lines) on 28 August 2026, checking for silent exception handling, mutable defaults, dynamic query construction, unguarded division, missing request timeouts, shared mutable state and function length. Finding A-1 was additionally reproduced by direct execution against `is_livelocked()`. Line references are to branch `raihan-working`.